

7

Laboratory 7

7	Optical flow	61
7.1	Main objectives	
7.2	Theory of optical flow	
7.3	Implementation of the block method	
7.4	Implementation of optical flow in multiple scales	
7.5	Other methods of determining optical flow	
7.6	Direction detection and object classification using optical flow	

7. Optical flow

7.1 Main objectives

- You will become familiar with the issue of optical flow,
- You will learn about the applicability of optical flow and its limitations,
- You will become familiar with the use of a multi-scale approach in determining optical flow,
- You will implement the block method,
- You will become familiar with the methods available in the OpenCV library,
- You will use the optical flow to segment moving objects.

7.2 Theory of optical flow

The optical flow is a vector field describing the movement of pixels (possibly only selected ones, i.e. feature points) between two consecutive frames from a video sequence. In other words, for each pixel on frame I_{n-1} we determine the displacement vector resulting in image I_n . It is worth noting, however, that the flow can be (and sometimes is even desirable) calculated for frames with a gap greater than '1'.

The optical flow can be *dense* or *sparse*. In the first case, the displacement is determined for each pixel in the image, and in the second only for a limited set of them.

The basis of optical flow operation is tracking pixels (or certain local pixel configurations, i.e. surroundings) between frames. This means that on subsequent frames we look for the same pixels (or their small surroundings). This assumes that the brightness of a pixel is constant (on a greyscale, in the general case – constant colour).

Unfortunately, as the exercise will show, the approach fails in several cases, the most important of which are large, homogeneous areas. Why does this happen? Imagine a rather large carton box. It has grey, homogeneous walls with no texture and is illuminated uniformly. We want to determine the optical flow for all pixels in the image, including the pixel from the centre of the box wall. Will we be able to find exactly this point on the next frame? Is an unambiguous assignment possible?

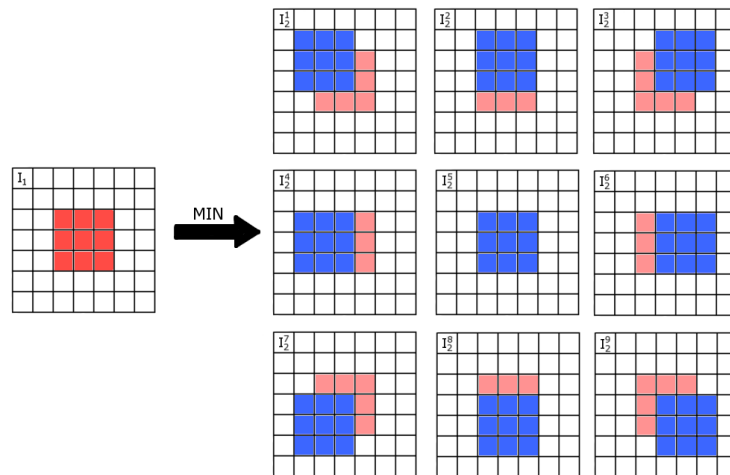


Figure 7.1: Example of searching for a matching block.

For this reason, it is sometimes preferable to use a sparse flow, calculated at predefined locations. Here we have two possibilities. Either we arbitrarily select points e.g. on a square or rectangular grid (then the aim is to reduce the computation time), or we search for so-called feature points – these can be edges, corners or other distinct image elements. There are multiple algorithms for their detection, e.g. Harris corner detector, SIFT or SURF feature point detectors etc.

A very large number of methods have been described in the literature – please see the rankings available at [Middlebury](#), [KITTI](#), [Sintel](#). The two most popular classical methods are Horn-Schunck (HS) and Lucas-Kanade (LK). The second one is available in the OpenCV library.

7.3 Implementation of the block method

The block method, as the name suggests, is based on matching some pixel blocks between successive frames. From one image, we crop blocks of a certain size (e.g., 3×3 , 5×5 , etc.) using the sliding window method across the image. For the second image, we look for the most similar block to the cropped one, with the search limited to a small neighbourhood of the cropped block's coordinates. There are two reasons for doing this – usually the displacement of pixels between successive frames is small, and we also significantly reduce the execution time of the algorithm.

Exercise 7.1 Implement the block method for determining the optical flow.

1. Create a new script and load two consecutive frames from the sequence, which we will call I and J (the earlier and later one, respectively), for example frames number 150 and 151. Optionally, you can downscale both images for testing purposes, e.g. by a factor of four – the computations will be faster. Convert the images to greyscale using `cvtColor`. Display the given images and their difference using `cv2.absdiff`.
2. Implement a block method for determining optical flow. Make the following assumptions – we compare image patches of size 7×7 . Further used symbols and values for the 7×7 window:
 - $W2 = 3$ – integer value from half the size of the window,
 - $dX = dY = 3$ – search size in both directions.

3. You should start your implementation with two `for` loops on the image. Use the `W2` parameter taking into account the edge case – we assume that we do not calculate the optical flow on the edges.
4. Inside the loop, we cut out a part of the `I` frame. As a reminder, the necessary syntax is: `I0 = np.float32(I[j-W2:j+W2+1,i-W2:i+W2+1])`. In the exercise we assume that `j` – outer loop index (by lines), `i` – inner loop index (by columns). Please note the „+1” component. Conversion to a floating point type is needed for further calculations.
5. Then we execute the next two `for` loops – searching around the pixel `J(j,i)`. They range from $-dX$ to dX and from $-dY$ to dY . Inside the loop, check that the coordinates of the current context (i.e. its centre) are within the allowed range. Alternatively, you can also modify the range of the outer loops to exclude access to pixels beyond the acceptable range.
6. We cut out `J0` surroundings, converting them to `float32` and then calculating the „distance” between `I0` and `J0` slices. This can be done with the instruction: `np.sqrt(np.sum((np.square(J0-I0))))`. From all the fragments of `J0` for a given `I0`, find the smallest „distance” – the location of the „closest” to the `I0` patch in the image `J`.
7. The obtained coordinates of the minima found should be written in two matrices (for example `u` and `v`) – they should be created before and initialised with zeros, the dimensions are like for `I`.
8. The determined field of the optical flow can be visualised in two ways – through the vectors (arrows) – the `plt.quiver` function from the `matplotlib` library or colours – we will implement the second approach. The idea is to find the angle and length of the vector defined by the two optical flow components `u` and `v` for each pixel. We will then get a data representation similar to the HSV colour space. When displayed, the colour indicates the direction in which the pixels move, while its saturation indicates the relative speed of pixel movement – analyse the colour wheel in Fig. 7.2.

Convert the determined flow to the polar coordinate system – `cartToPolar`. Create an image variable in HSV space with input image dimensions, 3 channels, and `uint8` type. The first channel is component `H`, equal to $angle \cdot 90 / np.pi$ (range from 0 to 180 in Python). The second channel is the `S` component – normalise the vector length to the range 0-255. The third channel is component `V`, set it to 255.

R To make no motion appear black instead of white, swap `S` and `V` channels.

Finally, convert the image to RGB space and display it.

9. Please analyse the results and experiment with the input image size and with the `W2`, `dX` and `dY` parameters (e.g. image reduced twice, parameters equal to 5). Do such parameters allow for the correct determination of the optical flow for the input image in original size? How large window is needed to get similar results?

R Due to its simplicity, the block method is quite inaccurate – the calculated horizontal or vertical shift is integer. In other methods (e.g. HS or LK), the optical flow is mostly decimal.

To reduce the number of incorrect matches, you can add some filtering of the results based on the previously computed brightness difference between frames.

Binarise it, then perform dilation (test different mask sizes and numbers of iterations). Finally, we “let through” only those flow results that are computed for pixels located in the difference image after dilation.

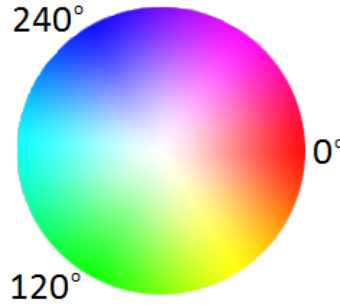


Figure 7.2: Colour wheel.

Example results for the analysed test sequence is shown in Figure 7.3.

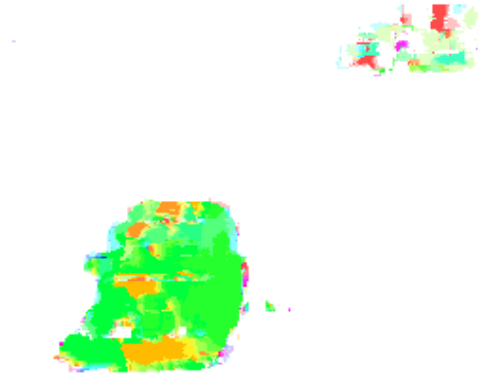


Figure 7.3: Example result of the optical flow determined by the block method for images I and J (frame numbers 150 and 151 from the *highway* sequence for parameters $W2 = dX = dY = 3$, window size 7×7 , and the original image resolution).

7.4 Implementation of optical flow in multiple scales

The detection of large displacements by the implemented method is computationally complex. This is due to the need to use large values for the parameters dX and dY , and therefore for most locations (except edge locations) it will be necessary to check the similarity of more contexts (more SAD operations). This approach is very inefficient.

A better, and very commonly used idea, is to process the image at multiple scales. The idea is shown in Figure 7.4.

Schematic of how the method works:

- perform scaling of images (I and J) to scales: L_0 – original resolution, $L_1, \dots, L_m$ (the smallest image) – this is referred to in the literature as an image pyramid. This type of solution usually uses scaling with a factor of 0.5.

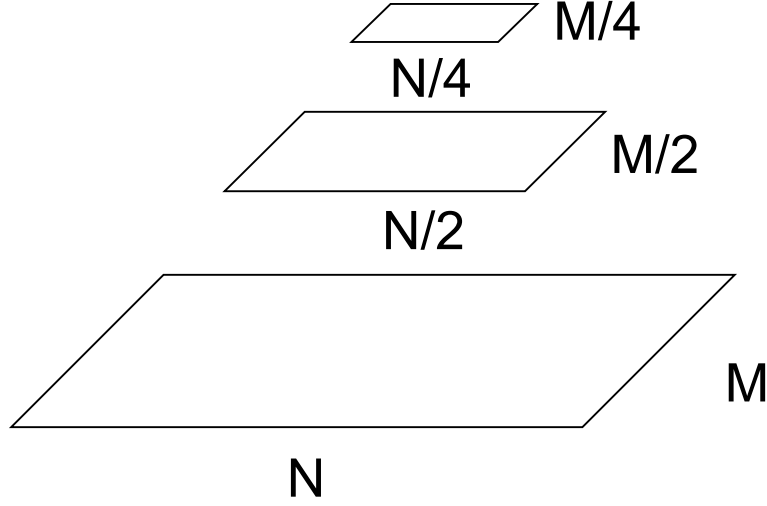


Figure 7.4: Example of pyramid image construction (multiple scales).

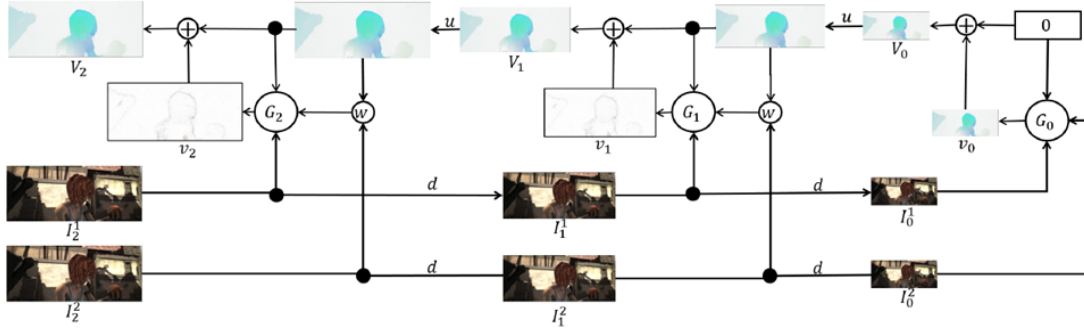


Figure 7.5: Performing multi-scale computations (in this case 3 scales) illustrated on the example of the SpyNet neural network <https://spynet.is.tue.mpg.de/>. G is the (arbitrary) flow estimation algorithm, w is the warping operation, v is the flow obtained at a given scale, V is the total flow summed over multiple scales, and u is the image (flow) after increasing its dimensions to a higher scale.

- calculate the optical flow for a pair of images I and J on a scale of L_m (e.g. using the block method).
- propagate the results of the calculated flow to the L_{m-1} scale:
 - in the first step, the obtained flow is scaled to the size of L_{m-1} , and the flow values themselves are also increased accordingly,
 - next, the previous image frame is modified according to the flow (so-called *warping*). The purpose of this operation is motion compensation, i.e. shifting the pixels in such a way that, at the coarser scale L_{m-1} , corresponding pixels in the two images are closer to each other.
- we calculate more accurate flow values at level L_{m-1} between image I and the modified image J (e.g. using the block method),
- we separately sum the flows obtained at different scales,
- the procedure is carried out up to level L_0 .

Compare the description with Figure 7.5.

Exercise 7.2 Implement the multi-scale version of the block method for optical flow calculation.

1. At the beginning, please create a copy of the algorithm you have created so far in Task 1. Next, we convert the part of the algorithm concerning the computation of the OF into **function** for the selected scale.

The function should look like this:

```
def of(J_org, I, J, W2=3, dY=3, dX=3):
```

The individual parameters are J_org and I – input images, J – image after modification, W2, dX, dY – method parameters (identical to 7.3). In fact, you don't need to pass J_org to the function – the suggested solution is to calculate **absdiff** between images and display the results and images J_org, I and J for a better understanding of how the method works. In fact, the calculations are for I and J. In the function, just like before, it is possible to perform additional binarisation and dilation.

2. It will also be useful to convert the part of the algorithm concerning the visualisation of the optical flow into a **function**. This function should be of the form:

```
def vis_flow(u, v, YX, name):
```

The individual parameters are u and v – optical flow components, YX – image dimensions, name – window name displayed, e.g. 'of scale 2'.

3. After writing the function, it is good to test its operation for one scale (L_0) – the result should be the same as the one obtained earlier.
4. After creating two functions based on the written algorithm, we proceed to write a function to generate a pyramid of images. The following function can be used:

```
def pyramid(im, max_scale):
    images=[im]
    for k in range(1, max_scale):
        images.append(cv2.resize(images[k-1], (0,0), fx=0.5, fy=0.5))
    return images
```

In this case, we assume that the resolution reductions will always be double. So the input image is downsampled 2 times, 4 times, 8 times, etc. For the purposes of the experiment, we can assume the limit of 2-3 scales. A pyramid should be generated for each of the input images.

5. We start processing from the smallest scale, so to the I variable we assign the smallest-scale pyramid image: $I = IP[-1]$, where IP is the generated pyramid for the first image. Please note the syntax $[-1]$ – access to the last item.
6. The key component of the algorithm is the scales loop – please consider the starting and ending indexes. We start by computing the flow and doubling its dimensions and values. The function: `cv2.resize((2**s)*u, (0,0), fx=2**s, fy=2**s, interpolation=cv2.INTER_LINEAR)`.
7. Make a copy of the second image J_new. The next step is to modify this copy according to the flow. We perform this operation for all scales except the largest one (i.e. an image with the input's dimensions). This can be done using two for loops with the protection to keep the indexes in the allowed range. To individual pixels from J_new we assign appropriate pixels from J, according to the calculated flow. It is worth checking if the second image after modification is similar to the first image (I) – if it is, the operation has been performed correctly. Finally, we assign the image J_new to J.
8. The last element of the algorithm is the calculation of the total flow and its

visualisation. To do this, prepare an empty 2D table for each of the u and v components with the dimensions of the input image. Then, in a loop over the scales, we add the flows from different scales to each other.

- R** A short example for clarification. We have a pixel whose displacement is 9 (*ground truth*), but our search range is limited to 5 pixels in each direction. We downscale the image by a factor of two; now the pixel's displacement should be 4.5. We find the best-matching pixel at the smaller scale – we obtain a displacement of 5, so at the larger scale the displacement would be 10. According to this result, we modify the image at the larger scale (that is, we shift the pixel by 10 in the appropriate direction). We then compute the flow again, this time at the larger scale, and obtain a displacement of -1 (the residual flow). Finally, we sum the results from both scales and obtain a total displacement of 9.

Conclusions. To get the correct final result, the u and v flows from each scale should be increased in two ways – the dimensions of these „images” must be the same as for the input I and the flow values must be increased 2, 4, 8 etc. times, depending on the scale. The summed flow from different scales should be visualised with the help of a function `vis_flow`.

9. Compare how the method works with and without multiple scales. Analyse the results for larger window sizes in one scale, and for smaller window sizes with multiple scales, pay attention to execution time. Was it possible to obtain the correct flow for a small window (e.g. $dX = dY = 3$) using a multi-scale method (e.g. 2-3 scales) for an image of original size (i.e. for a larger pixel displacement)?

- R** The multi-scale method works very well in theory – in practice, pixel matching is not always accurate, and scaling down and up always results in the loss of some information, so that even minor errors are propagated to subsequent scales and the final result may be partially incorrect or noisy. For this reason, in practice, 2, 3, or a maximum of 4 scales are used (but nevertheless, the results still differ slightly from the „expected” ones).

In Figures 7.6 and 7.7, the warped image and the total optical flow for 2 scales are shown, respectively.



Figure 7.6: Image after modification (warping).



Figure 7.7: Total optical flow summed over 2 scales.

7.5 Other methods of determining optical flow

Use the script created in the previous exercises to read the image sequence. Make sure that it includes the parameter `iStep`, which allows you to set how many frames are skipped during processing.

R For simplicity, sequences in greyscale will be analysed.

Several functions are available in the OpenCV library to calculate optical flow:

- Lucas-Kanade (multi-scale, sparse and dense),
- Farneback,
- Dual TV-L1,
- PCA Flow,
- DIS Flow,
- Simple Flow,
- Deep Flow.

R Details of the specific algorithms are available in the [OpenCV documentation](#). It is also worth taking a look at the [optical flow tutorial](#) and analysing it.

Exercise 7.3 Run sample solutions.

1. Task 1 – determine the optical flow by dense methods (i.e. all but sparse LK). The code should consist of several elements:

- loading of input images and conversion to greyscale,
- creating a variable `flow` with two channels (for `u` and `v`),
- creating an instance of the given optical flow determination method and calling `calc` on it,
- visualisation of results as in chapter 7.3.

Note the results for different methods (including the accuracy of determining the edges of objects), as well as the time needed to perform the calculations.

R For visualising dense LK, it may be useful to restrict the flow modulus to the interval 0-10, e.g. by `mag[mag>10]=10`.

2. Task 2 – run a sparse optical flow using the Lucas-Kanade method. The [tutorial](#) shows the tracking of feature points. We will calculate the optical flow for uniformly distributed points. So the code needs to be adapted accordingly. Some remarks:

- the algorithm has a number of parameters, which should be clear after the analysis of the lecture (possibly read in the documentation) – size of the window, number of scales, criteria for terminating the calculation (solving the system of equations),
- generate a grid of uniformly spaced points (e.g. a step of 10 pixels) – here it is useful to preview the form of `p0` from the example,
- the second difficulty is a different visualisation. Example solution:

```
img = I
for jj in range(0, y, 1):
    if (st[jj] == 1):
        cv2.line(img, (points[jj,0,0],points[jj,0,1]), (new_points
            [jj,0,0],new_points[jj,0,1]), (0,0,255))
```

- please do not forget to assign the current image to the previous one at the end of the loop.

Please observe how the algorithm works – are the determined flows correct? You can add a vector direction to the display – mark the start or end of the vector with a dot. You can also remove vectors with a small modulus.

Example results for the analysed test sequences are shown in Figure 7.8.

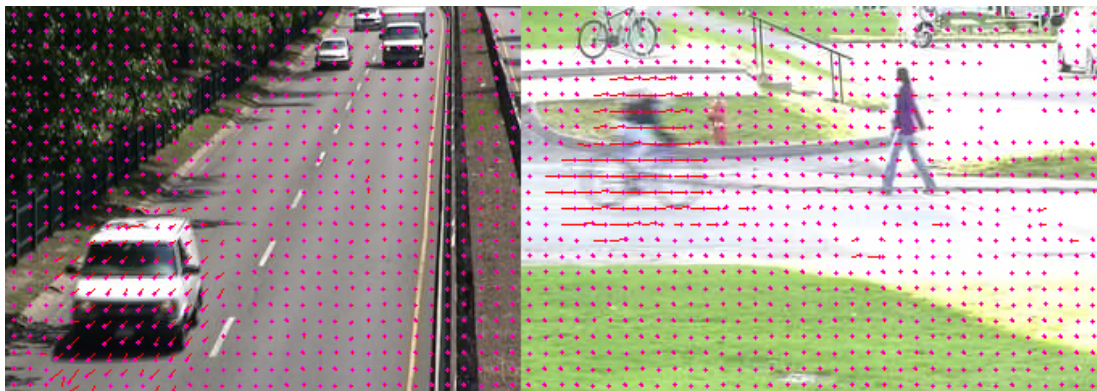


Figure 7.8: Example results of sparse optical flow determined by the LK method: left for image index 158 from the *highway* sequence; right for image index 471 from the *pedestrians* sequence.

7.6 Direction detection and object classification using optical flow

The idea is as follows. A car, which is a rigid solid, should be characterised by coherent motion – each of its elements should have similar displacement, at least in terms of direction. A walking person is a contrary, as the specifics of walking cause the limbs to move in different directions (depending on the phase of the walk).

So we will try to analyse the motion vectors for each object (depending on the sequence of the car or the human) and see if we can tell something from this and possibly make a classification.

Exercise 7.4 Non-obligatory assignment.

Implement the algorithm, based on the optical flow program from the previous task (e.g. Farneback's algorithm).

1. First we will add foreground object segmentation. Use the MOG/GMM algorithm used in previous labs.
2. A very important issue is mask filtration. Here again, the experience gained from previous exercises will be useful. It seems to be a good idea to disable shadow detection – `detectShadows=False` in the MOG2 constructor.
3. In the next step we add a labelling, the result of which we display.
4. We then perform an optical flow analysis. It comes down to calculating the mean and standard deviation for the modulus and angle of the vectors corresponding to a given object. The difficulty of the task depends on your level of skill in Python.

A few hints (you may or may not want to use them):

- we carry out calculations only if there are objects – `retval > 0`,
- the key is a convenient container for averages and variances. You can use lists and the `append` instruction. Then we will get the functionality as in Matlab. It is easier to use two separate lists, as it is simpler to calculate the statistics later.
- we perform the essential calculations in a loop on the image. If the label is different from zero then:
 - check whether the amplitude of the optical flow at this point is greater than a given threshold (e.g. 1),
 - if so, calculate the angle of the vector (`atan2` from `math`) and add the result to our list at the appropriate address.
- in the next step we calculate the mean and standard deviation for the modulus and angle. The functions `mean` and `stdev` from the `statistics` package will be useful.

 Before computing, you need to check that there are at least two elements in the list for the object.

- the final stage is visualisation. The easiest way to adapt the code from the moving object segmentation exercise is to display the determined parameters – two averages and two standard deviations. Optionally you can add filtering of small objects.
5. Analysis of results:
 - How do the mean and standard deviation for both sequences behave?
 - Does the average direction of movement correspond more or less to reality?
 - Is it possible to see any difference between the two sequences – standard deviation?
 - Are there problems with the segmentation? What kind of problems?

Alternatively, you could try displaying the mean motion vector for each object.

Example results for the analysed test sequences are shown in Figure 7.9 and 7.10.

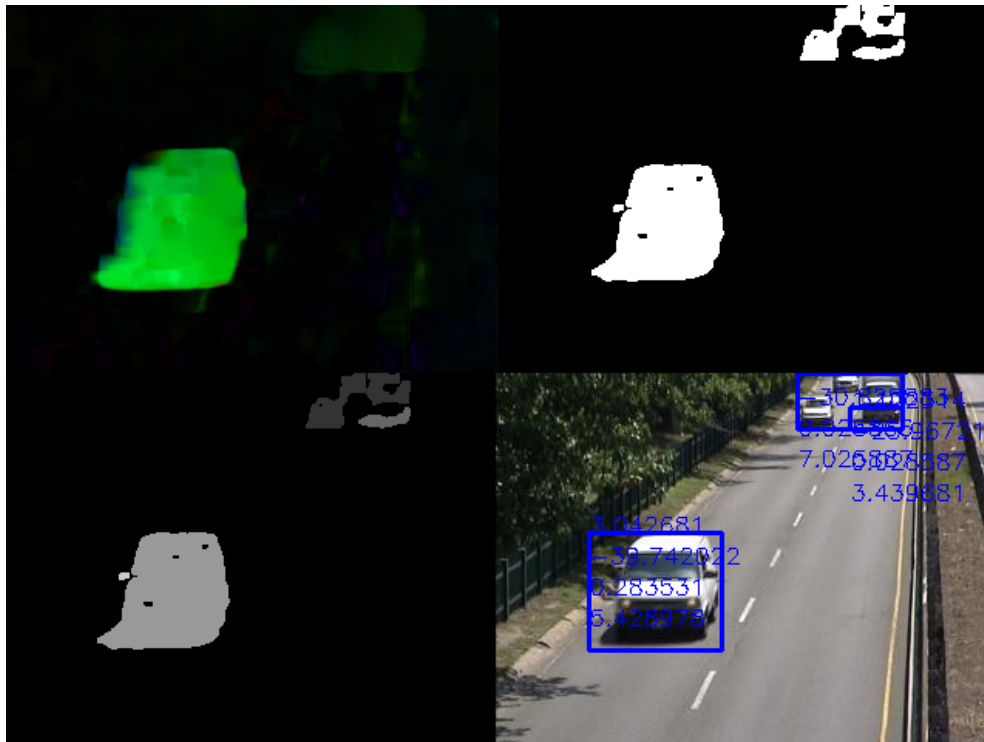


Figure 7.9: Example results for an image with index 146 from the *highway* sequence: top left – Farneback optical flow, top right – foreground object segmentation, bottom left – labelling, bottom right – detected objects with calculated parameters.

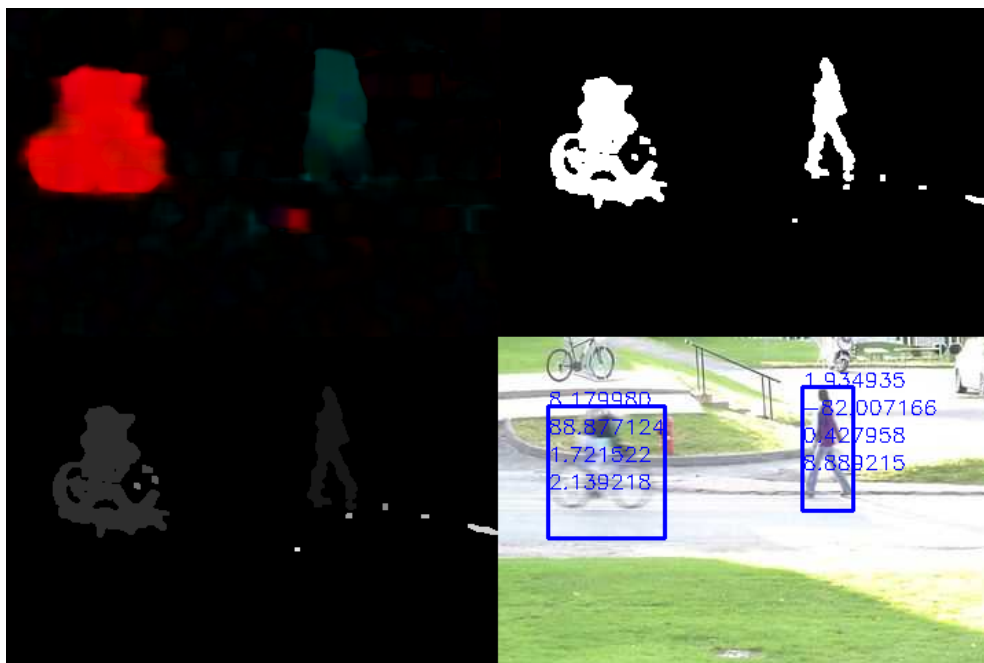


Figure 7.10: Example results for the image with index 471 from the *pedestrians* sequence: top left – Farneback optical flow, top right – foreground object segmentation, bottom left – labelling, bottom right – detected objects with calculated parameters.